PRACTICAL REPORT

Submitted to the

Department of Artificial Intelligence

In Partial Fulfillment of the Requirement for the Degree of

Bachelor of Technology

(Artificial Intelligence)

Entitled

"Spatio-Temporal Air Pollution Forecasting Using Graph Neural Networks and LSTM"

Subject: Big Data Analytics (AI358)

Course Coordinator: Dr. Pruthwik Mishra

: Submitted By :

Miten Gandhi	U23AI017
Harshil Andhariya	U23AI021
Vatsal Bateriwala	U23AI030
Aksh Dharodiya	U23AI031

B.TECH. 3rd Year, 6th Semester

(Year: 2025–26)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

Sardar Vallabhbhai National Institute of Technology

Surat-395007, Gujarat, INDIA.

Sardar Vallabhbhai National Institute of Technology

Surat-395 007, Gujarat, INDIA.

Department of Artificial Intelligence

CERTIFICATE

This is to certify that the **Practical Report** entitled **"Spatio-Temporal Air Pollution Forecasting Using Graph Neural Networks and LSTM"** has been presented and submitted by the following candidates of B.Tech. 6th Semester, in partial fulfillment of the requirement for the award of the **B.Tech.** degree in **Artificial Intelligence**, Department of Artificial Intelligence, for the academic year **2025–26**.

The subject of this report is **Big Data Analytics (AI358)** under the guidance of **Dr. Pruthwik Mishra**, Department of Artificial Intelligence, SVNIT Surat.

Sr. No.	Name of Student	Enrollment No.
1	Miten Gandhi	U23AI017
2	Harshil Andhariya	U23AI021
3	Vatsal Bateriwala	U23AI030
4	Aksh Dharodiya	U23AI031

They have successfully and satisfactorily completed their practical report in all respects. We certify that the work is comprehensive, complete and fit for evaluation.

Dr. Pruthwik Mishra
Assistant Professor
Dept. of Artificial Intelligence
SVNIT, Surat

Ms. Aparna Tripathi
Teaching Assistant
Dept. of Artificial Intelligence
SVNIT, Surat

ABSTRACT

Air pollution constitutes one of the most critical environmental challenges of the modern era, with fine particulate matter (PM2.5) posing substantial risks to human health and responsible for approximately 7 million premature deaths annually worldwide. Accurate, real-time forecasting of pollutant concentrations is inherently a spatio-temporal problem: readings at any monitoring station are influenced not only by the station's own historical values, but also by neighboring stations through physical transport mechanisms including wind advection, turbulent diffusion, and proximity to shared emission sources.

This project develops a complete, production-grade end-to-end pipeline for spatio-temporal PM2.5 forecasting, combining Graph Neural Networks (GNN) with Long Short-Term Memory (LSTM) networks applied to the United States Environmental Protection Agency's Air Quality System (AQS) dataset spanning 2022–2023. The EPA AQS dataset is a genuine *big data* asset: it contains over **12.4 million hourly PM2.5 readings** for the two-year window alone, and over **2.1 billion rows** across all pollutants and all years from 1978 to 2025 — a scale that fundamentally requires distributed processing infrastructure rather than single-machine tools.

The pipeline comprises five principal stages: **(i)** distributed ingestion, extraction, cleaning, and feature engineering of the raw EPA CSV archives using Apache Spark (PySpark) with a 10 GB distributed memory configuration; **(ii)** scalable time-series persistence in HBase via the HappyBase Python client, with a compound row-key design `station_id#YYYYMMDD#HH` enabling sub-millisecond range scans; **(iii)** spatial graph construction over 805 well-covered monitoring stations using a BallTree Haversine index with a 150 km connectivity radius, producing a graph of 805 nodes and 10,928 directed edges; **(iv)** training of an ST-GNN-LSTM model with 26.6 million parameters combining Residual GCN layers for spatial feature extraction with a 2-layer LSTM for temporal sequence modeling, trained using Huber loss with early stopping on Apple M4 Pro MPS hardware; and **(v)** a comprehensive visualization suite in R using ggplot2, leaflet, plotly, and ggridges, producing eight publication-quality outputs including interactive geospatial maps and multi-step forecast dashboards.

The model achieved a best validation Huber loss of 0.02336, converging in 9 epochs (67.2

minutes of training time). Test-set evaluation revealed a systematic positive bias of $+39.714\ \mu$g/m^3 — a characteristic distributional shift artifact in spatio-temporal forecasting that motivates specific improvements discussed in the conclusion.

Keywords: Spatio-Temporal Forecasting, Graph Neural Network, LSTM, PM2.5, Apache Spark, HBase, HappyBase, PyTorch Geometric, Air Quality, Big Data, EPA AQS, GCN, Residual Network

TABLE OF CONTENTS

1 Introduction **1**

1.1 Motivation and Problem Statement 1

1.2 Why Graph Neural Networks and LSTM? 2

1.2.1 The Spatial Structure of Pollution Networks 2

1.2.2 The Temporal Structure of Pollution Time Series 2

1.3 Project Objectives 3

1.4 Report Organization 4

2 Why Big Data? The Case for Distributed Processing **5**

2.1 The Full Scale of the EPA AQS Dataset 5

2.2 The Four V's Applied to EPA AQS 6

2.2.1 Volume — Scale that Exceeds Single-Machine Capacity 7

2.2.2 Velocity — Continuous, High-Frequency Data Ingestion 8

2.2.3 Variety — Heterogeneous Multi-Pollutant, Multi-Source Data . . . 8

2.2.4 Veracity — Data Quality at Scale 9

2.3 Scalability: From 2 Years to 47 Years 9

2.4 Why HBase for Storage? 10

3 Dataset Description **12**

3.1 EPA Air Quality System (AQS) Overview 12

3.2 Data Format and Schema 13

3.3 PM2.5 Distribution and Statistical Characteristics 14

3.4 Multi-Pollutant Dataset Scale 16

3.5 Temporal Patterns 17

3.6 Geographic Distribution 18

3.7 State-Level Analysis 18

4 Pipeline Architecture **20**
4.1 System Overview . . . 20
4.2 Stage 1 — Distributed Data Ingestion with Apache Spark . . . 20
4.2.1 Configuration and Initialization . . . 20
4.2.2 ZIP Extraction and Multi-Year Loading . . . 21
4.2.3 Data Cleaning Operations . . . 21
4.2.4 Distributed Feature Engineering via Window Functions . . . 21
4.2.5 Station Filtering and Parquet Persistence . . . 22
4.3 Stage 2 — HBase Time-Series Storage via HappyBase . . . 22
4.3.1 HBase Architecture and Row Key Design . . . 22
4.3.2 Column Family Schema . . . 23
4.4 Stage 3 — Spatial Graph Construction . . . 23
4.4.1 BallTree Haversine Index . . . 23
4.4.2 Edge Weight Computation and Normalization . . . 24

5 Model Architecture and Training **25**
5.1 STGNN-LSTM Architecture Overview . . . 25
5.2 Residual GCN Layer . . . 26
5.3 Batched Graph Expansion . . . 26
5.4 LSTM Temporal Encoder . . . 26
5.5 Training Configuration . . . 28
5.5.1 Loss Function: Huber Loss . . . 29
5.5.2 Autocorrelation Analysis . . . 29
5.5.3 Training Progression . . . 30

6 Experimental Results and Visualization **31**
6.1 Quantitative Evaluation Metrics . . . 31
6.2 Forecast vs. Actual Analysis . . . 31
6.3 Multi-Step Recursive Forecasting . . . 32
6.4 R Visualization Suite . . . 33

7 Limitations and Future Work **35**
7.1 Identified Limitations . . . 35

7.1.1 Scaler Data Leakage (Primary) 35
7.1.2 Architecture Over-Parameterization 35
7.1.3 Static Graph Topology 35
7.2 Future Work Directions 35

8 Conclusion **37**

INTRODUCTION

1.1 Motivation and Problem Statement

Air pollution is responsible for approximately 7 million premature deaths annually according to the World Health Organization, making it the single largest environmental health risk on the planet. Among all regulated air pollutants, fine particulate matter — particles with aerodynamic diameter below 2.5 micrometers (PM2.5) — is identified as the most health-damaging. PM2.5 penetrates deep into the pulmonary alveoli, crosses into the bloodstream, and is directly associated with cardiovascular disease, stroke, chronic obstructive pulmonary disease, and lung cancer. Unlike coarser particles, PM2.5 remains suspended in the atmosphere for days to weeks, allowing it to travel hundreds of kilometers from its emission sources and affect populations far removed from industrial zones.

The economic burden is equally staggering. The World Bank estimates that air pollution costs the global economy over $5 trillion annually in welfare losses, with developing nations bearing a disproportionate share. In the United States alone, the EPA estimates that achieving PM2.5 attainment standards prevents approximately 4.2 million premature deaths and generates $30 in economic benefit for every $1 spent on air quality enforcement.

Accurate short-term forecasting of PM2.5 concentrations enables:

- Timely public health advisories (e.g., "Air Quality Index: Unhealthy — avoid outdoor exercise")
- Regulatory enforcement and non-attainment identification
- Urban traffic management to reduce peak emission windows
- Hospital preparedness for respiratory emergency surges
- Industrial permitting and curtailment decisions

Forecasting air quality, however, is fundamentally a *spatio-temporal* problem. Pollution at a monitoring station is not self-contained: wind transport, topographic channeling, and proximity to shared emission sources mean that neighboring stations carry substantial predictive information about a target station's future concentrations. A wildfire plume originating in

California will register at Oregon stations hours before it reaches Washington — information that is completely invisible to a per-station univariate model. Traditional approaches (ARIMA per-station, standalone LSTM) ignore this spatial correlation and consequently underperform compared to models that jointly represent the full monitoring network.

1.2 Why Graph Neural Networks and LSTM?

1.2.1 The Spatial Structure of Pollution Networks

Monitoring station networks are naturally represented as irregular graphs: nodes correspond to stations, and edges encode spatial relationships such as geographic proximity, wind-direction alignment, or shared airshed membership. Unlike image data (which sits on a regular pixel grid amenable to CNNs) or sequence data (which maps to RNNs), monitoring networks are irregular, non-Euclidean structures of variable size and topology. Graph Neural Networks (GNNs) are precisely designed for this setting, performing learnable information aggregation from each node's neighbors via message-passing:

$$\mathbf{h}_v^{(l+1)} = \sigma\left(\mathbf{W}^{(l)} \cdot \text{AGGREGATE}\left(\left\{\mathbf{h}_u^{(l)} : u \in \mathcal{N}(v)\right\}\right) + \mathbf{b}^{(l)}\right) \tag{1.1}$$

where $\mathcal{N}(v)$ is the set of neighbors of node v, $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are learnable parameters at layer l, and σ is an activation function. The GCN variant (Kipf & Welling, 2017) aggregates with normalized adjacency:

$$\mathbf{H}^{(l+1)} = \sigma\left(\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right) \tag{1.2}$$

where $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ (adjacency with self-loops) and $\tilde{\mathbf{D}}$ is its degree matrix.

1.2.2 The Temporal Structure of Pollution Time Series

PM2.5 exhibits strong temporal autocorrelation at multiple scales: hourly (traffic-driven diurnal cycle), daily (boundary layer dynamics), weekly (weekday vs. weekend traffic patterns), and seasonal (winter inversion layers, summer ozone season). Long Short-Term Memory (LSTM) networks are a well-established architecture for capturing such multi-scale temporal dependencies, using gated memory cells that learn *what to remember* and *what to forget* across arbitrary-length sequences:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f) \quad \text{(forget gate)} \tag{1.3}$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i) \quad \text{(input gate)} \tag{1.4}$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}_c \mathbf{h}_{t-1} + \mathbf{b}_c) \quad \text{(cell state)} \tag{1.5}$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{b}_o) \quad \text{(output gate)} \tag{1.6}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad \text{(hidden state)} \tag{1.7}$$

The combination of GNN (spatial) + LSTM (temporal) in the STGNN architecture is therefore a principled design choice: the GCN at each timestep produces spatially-enriched node representations, which the LSTM then sequences across time to capture how the neighborhood context evolves.

1.3 Project Objectives

The primary objectives of this project are:

1. To demonstrate a genuine **big data pipeline** by ingesting and processing the EPA AQS dataset (12.4M+ PM2.5 rows; 2.1+ billion rows at full historical scale) using Apache Spark and HBase, with explicit quantification of why single-machine tools are insufficient.
2. To design and implement a **scalable HBase storage layer** with a compound row-key scheme optimized for time-series range scans over monitoring station histories.
3. To construct a **spatial graph** over 805 US monitoring stations using BallTree Haversine indexing and train an ST-GNN-LSTM model on PM2.5 time-series data.
4. To **evaluate** the model against persistence baselines and conduct detailed error analysis using both per-station and aggregate metrics.
5. To produce **comprehensive visualizations** using R, including interactive geospatial maps (Leaflet), multi-step forecast dashboards (ggplot2), and seasonal pattern analyses (ggridges, plotly).

1.4 Report Organization

Chapter 2 presents the big data justification in full detail. Chapter 3 describes the EPA AQS dataset and its characteristics. Chapter 4 covers the complete system architecture and pipeline stages. Chapter 5 details the GNN+LSTM model. Chapter 6 presents experimental results. Chapter 7 discusses limitations and future work. Chapter 8 concludes.

WHY BIG DATA? THE CASE FOR DISTRIBUTED PROCESSING

This chapter provides the *primary technical justification* for why this project is genuinely a Big Data problem and why tools like Apache Spark and HBase are not merely pedagogically convenient but technically *necessary*.

2.1 The Full Scale of the EPA AQS Dataset

The EPA Air Quality System has been continuously operating since 1978, aggregating hourly readings from thousands of monitoring stations across all US states and territories. The dataset is not static: it grows by approximately 180 million new rows per year. Table 2.1 presents the complete scale of the EPA AQS dataset.

Pollutant	Years available	Approx. rows (full)	2022–23 only
PM2.5 FRM (88101)	2000–2025	∼310 million	12.4 million
PM2.5 non-FRM (88502)	2004–2025	∼85 million	5.7 million
PM10 Mass (81102)	1978–2025	∼270 million	7.3 million
PMc Mass (86101)	1988–2025	∼105 million	2.6 million
PM2.5 Speciation	1999–2025	∼290 million	9.6 million
PM10 Speciation	2000–2025	∼380 million	14.1 million
Ozone (44201)	1978–2025	∼400 million	9.0 million
SO_2 (42401)	1978–2025	∼250 million	3.1 million
CO (42101)	1978–2025	∼180 million	1.9 million
NO_2 (42602)	1978–2025	∼220 million	3.6 million
TOTAL (all pollutants)	**1978–2025**	**∼2.49 billion**	**∼69.3 million**

Table 2.1: Full scale of the EPA AQS dataset from 1978 to 2025. This project used only the 2022–2023 window (69.3 million rows across all pollutants loaded into Spark), yet even this two-year subset exceeds the comfortable operating capacity of pandas on standard hardware.

Key Insight

Even a *single two-year window* of the EPA AQS dataset (69.3 million rows across all pollutants, 12.4 million for PM2.5 alone) approaches the memory limit of standard 16 GB laptops when loaded into pandas DataFrames with multi-column feature engineering. The *full historical dataset* of ∼2.5 billion rows is categorically impossible to process on a single machine.

2.2 The Four V's Applied to EPA AQS

The Big Data framework is commonly characterized by four dimensions: Volume, Velocity, Variety, and Veracity. The EPA AQS dataset satisfies all four to a degree that mandates distributed processing.

2.2.1 Volume — Scale that Exceeds Single-Machine Capacity

The core volume argument is both theoretical and empirical.

Theoretical analysis: A single row in the EPA AQS dataset contains approximately 24 columns. Loaded into a pandas DataFrame with standard dtypes (float64 for measurements, object for strings, datetime64 for timestamps), each row occupies approximately 300–400 bytes of RAM. For the full 2.5 billion row dataset:

$$\text{Memory required} = 2.5 \times 10^9 \text{ rows} \times 350 \text{ bytes/row} = 875 \text{ GB} \tag{2.1}$$

This is approximately 55× the RAM of a typical 16 GB laptop and 11× the RAM of a high-end 80 GB workstation. Even the two-year (2022–2023) slice across all pollutants requires:

$$\text{Memory (2022–23)} = 6.93 \times 10^7 \text{ rows} \times 350 \text{ bytes/row} \approx 24.3 \text{ GB} \tag{2.2}$$

This already exceeds a standard 16 GB laptop. When feature engineering adds rolling windows, lag columns, and cyclical encodings — roughly doubling the column count — the in-memory footprint grows to ∼50 GB for just two years of multi-pollutant data.

Empirical comparison: The following table compares what happens when you attempt to process the PM2.5 dataset with pandas versus Spark:

Operation	pandas (single machine)	PySpark (distributed)
Load 12.4M rows	∼18 min, 8 GB RAM	∼3 min, partitioned
Window lag features (24-col)	Memory Error (16 GB)	Scalable, ∼12 min
Group-by 805 stations	∼4 min	∼45 sec (parallel)
Write Parquet	Limited compression	5× compression, fast
Scale to full 1978–2025	**Impossible**	Linear scale-out

Table 2.2: Comparative processing capability: pandas vs. PySpark for the EPA AQS dataset.

2.2.2 Velocity — Continuous, High-Frequency Data Ingestion

The EPA AQS ingests data at near-real-time from thousands of automated monitoring stations. Each station reports one reading per hour, producing:

$$\text{New rows/year} = 3{,}000 \text{ stations} \times 365 \times 24 \text{ hours} \times 10 \text{ pollutants} \approx 263 \text{ million rows/year} \tag{2.3}$$

In a production forecasting system, new station readings must be ingested, feature-engineered, and made available for model inference within the hour they are reported. A pandas-based pipeline would be perpetually backlogged at this arrival rate. Spark Streaming (or micro-batch processing) provides the throughput to keep up with this ingestion rate while simultaneously running analytical queries over the historical archive.

2.2.3 Variety — Heterogeneous Multi-Pollutant, Multi-Source Data

The EPA AQS is not a single uniform dataset. It includes:

- **8 distinct pollutant families** with different units, measurement methods, and station coverage
- **Multiple measurement methods per pollutant** (e.g., FRM vs. non-FRM for PM2.5,

different analyzer types for ozone)

- **Meteorological data** (wind speed, wind direction, temperature, relative humidity) from co-located sensors
- **Categorical metadata** (state, county, monitor type, method code, data qualifier flags)
- **Sparse coverage** — not all stations measure all pollutants, and coverage varies dramatically by year

Joining these heterogeneous sources at scale requires a distributed SQL engine. Spark's DataFrame API with `join` and `union` operations executes these joins in a distributed, memory-efficient manner that pandas cannot replicate for data of this dimensionality.

2.2.4 Veracity — Data Quality at Scale

EPA AQS data contains substantial quality challenges that compound at scale:

- **Missing values**: Station outages, instrument calibration periods, and transmission failures produce gaps of 2–72 hours per station per year
- **Sensor malfunctions**: Erroneous readings (negative values, physically implausible spikes) must be identified and filtered
- **Data qualifier flags**: Readings marked with qualifier codes (e.g., `AQ` for "exceptional event") require specialized handling
- **Duplicate entries**: Multiple methods (FRM and non-FRM) reporting at the same station-timestamp pair create duplicates that must be deduplicated

Identifying and correcting these issues across 12.4 million rows requires Spark's distributed filtering, window function-based imputation, and deduplication operations. Applied to the full 2.5 billion row archive, the quality assurance step alone would run for days on a single machine.

2.3 Scalability: From 2 Years to 47 Years

This project used 2022–2023 data for model development. However, the scientific value of longitudinal air quality analysis requires the full historical archive. Consider what becomes possible with the complete 1978–2025 dataset:

- **Long-term trend detection**: Quantify the impact of the 1990 Clean Air Act Amend-

ments on PM2.5 trajectories across 30+ years

- **Event attribution**: Identify the spatial signature of major wildfires, dust storms, and industrial incidents across all years
- **Climate interaction modeling**: Correlate decadal PM2.5 trends with El Niño/La Niña cycles, drought indices, and temperature anomalies
- **Policy counterfactual analysis**: Compare PM2.5 trajectories in counties that attained NAAQS standards versus those that did not
- **Mortality impact estimation**: Estimate avoided deaths attributable to emissions reductions over the regulatory period

None of these analyses are feasible with single-machine tools. The Spark pipeline developed in this project is *directly extensible* to the full historical archive — one changes only the year loop in the data loading stage. The HBase storage schema, graph construction, and model architecture all scale without modification.

Scenario	Years	Approx. PM2.5 rows	Feasibility
This project	2022–23	12.4 million	pandas (barely) / Spark
5-year analysis	2019–23	∼31 million	Spark required
Post-CAA analysis	1990–2025	∼220 million	Spark required
Full archive	1978–2025	∼310 million	Spark cluster required
All pollutants, full archive	1978–2025	∼2.49 billion	Distributed cluster essential

Table 2.3: Scalability analysis: feasibility of different temporal scopes with and without distributed computing.

2.4 Why HBase for Storage?

Relational databases (PostgreSQL, MySQL) and file-based systems (CSV, pandas DataFrames) are unsuitable for the time-series access patterns required in an operational air quality forecasting system:

- **Range scan efficiency**: A forecasting model needs the last 12 hours of readings for all 805 stations simultaneously. In a relational database, this is a full table scan

with a `WHERE timestamp > NOW() - INTERVAL 12 HOURS` filter, which becomes slow at hundreds of millions of rows. HBase's row-key design places all readings for a station in lexicographic order, enabling sub-millisecond range scans via the `Scan(row_start, row_stop)` API.

- **Horizontal scalability**: As station count grows from 805 to 3,000+ (if all EPA monitors are included) or to a global network, HBase scales linearly by adding RegionServers without schema changes.
- **Column family separation**: Separating raw measurements (`cf_raw`), engineered features (`cf_features`), and metadata (`cf_meta`) into distinct column families allows selective column family loading — the model inference stage reads only `cf_features`, skipping the raw measurement family entirely.
- **Write throughput**: HBase sustains millions of writes per second via its write-ahead log (WAL) and MemStore architecture, enabling real-time ingestion from all 3,000+ EPA monitors simultaneously.

Summary: Why This Is Genuinely a Big Data Project

The EPA AQS dataset spans 47 years (1978–2025), contains approximately 2.5 billion total rows, grows by 263 million rows/year, and arrives from 3,000+ heterogeneous monitoring stations measuring 10+ pollutants. Even the two-year (2022–2023) slice used in this project — 69.3 million rows across all pollutants — exceeds the comfortable processing capacity of standard single-machine tools. Spark is used not as a pedagogical exercise but as a *technical necessity*: it is the only tool that can process this data within a reasonable time budget, and the only one that would scale to the full historical archive for longitudinal analysis.

DATASET DESCRIPTION

3.1 EPA Air Quality System (AQS) Overview

The United States Environmental Protection Agency's Air Quality System (AQS) is the most comprehensive publicly available air quality monitoring dataset in the world. It was established under the authority of the Clean Air Act and has operated continuously since 1978. The AQS aggregates hourly readings from approximately 3,000–4,000 monitoring stations across all US states and territories, covering the six criteria pollutants regulated under National Ambient Air Quality Standards (NAAQS): PM2.5, PM10, ozone, carbon monoxide, nitrogen dioxide, and sulfur dioxide.

The AQS data undergoes rigorous quality assurance before publication: readings are flagged, reviewed, and validated by state, local, and tribal air agencies (SLTTs) before submission to the EPA. Data is published annually as compressed ZIP archives, one file per pollutant per year, available at https://aqs.epa.gov/aqsweb/airdata/download_files.html.

Property	Details
Source	US EPA Air Quality System (AQS)
Available years	1978–2025 (47 years; ongoing)
This project	2022–2023 (24 months)
Temporal resolution	Hourly (one reading per station per pollutant per hour)
Primary pollutant	PM2.5 FRM/FEM (parameter codes 88101, 88502)
Other pollutants loaded	Ozone (44201), SO_2 (42401), CO (42101), NO_2 (42602), PM10 (81102), PMc (86101)
Total rows loaded (all pollutants)	∼69.3 million
PM2.5 rows (post-filter)	12,489,096
Stations (post-filter)	805
Coverage threshold	≥6,000 hourly readings (≈250 days)
Download URL	https://aqs.epa.gov/aqsweb/airdata/download_files.html

Table 3.1: EPA AQS Dataset Summary for this project

3.2 Data Format and Schema

Each EPA AQS annual file is a ZIP-compressed CSV with a consistent 24-column schema. The columns most relevant to this project are described in Table 3.2.

Column	Type	Description
State Code + County Code + Site Num	String	Three-part station identifier; combined to form `station_id`
Latitude / Longitude	Double	Geographic coordinates (WGS84)
Date Local / Time Local	String	Local-time observation date and hour
Sample Measurement	Double	Pollutant concentration in reported units
Units of Measure	String	Unit string (e.g., "Micrograms/cubic meter")
Parameter Name	String	Human-readable pollutant name
Qualifier	String	Data quality flag (e.g., blank = valid, "AQ" = wildfire)
State Name / County Name	String	Geographic labels for aggregation and display

Table 3.2: Key columns in the EPA AQS hourly data files

3.3 PM2.5 Distribution and Statistical Characteristics

PM2.5 concentrations in the 2022–2023 dataset exhibit a strongly right-skewed distribution characteristic of atmospheric pollutants. The majority of readings (approximately 85%) fall below the EPA 24-hour standard of 35 μg/m^3; however, extreme values reaching 497.6 μg/m^3 are present, reflecting wildfire smoke events concentrated primarily in western states during fire season (July–October).

Statistic	Value
Total readings (PM2.5, post-filter)	12,489,096
Mean	8.333 μg/m^3
Median	6.1 μg/m^3
Standard deviation	8.769 μg/m^3
25th percentile	2.8 μg/m^3
75th percentile	11.1 μg/m^3
95th percentile	26.3 μg/m^3
99th percentile	47.2 μg/m^3
Maximum	497.6 μg/m^3
EPA 24h NAAQS standard	35.0 μg/m^3
Exceedance rate	$\approx$4.2% of hourly readings
WHO annual guideline	5.0 μg/m^3

Table 3.3: Spark-computed PM2.5 summary statistics (2022–2023 combined, 805 stations)

These statistics are computed entirely in Spark — the `describe()` and `approxQuantile()` APIs execute distributed aggregations across all 12.4 million rows without loading the full dataset into driver memory. Figure 3.1 presents the full PM2.5 exploratory data analysis panel.

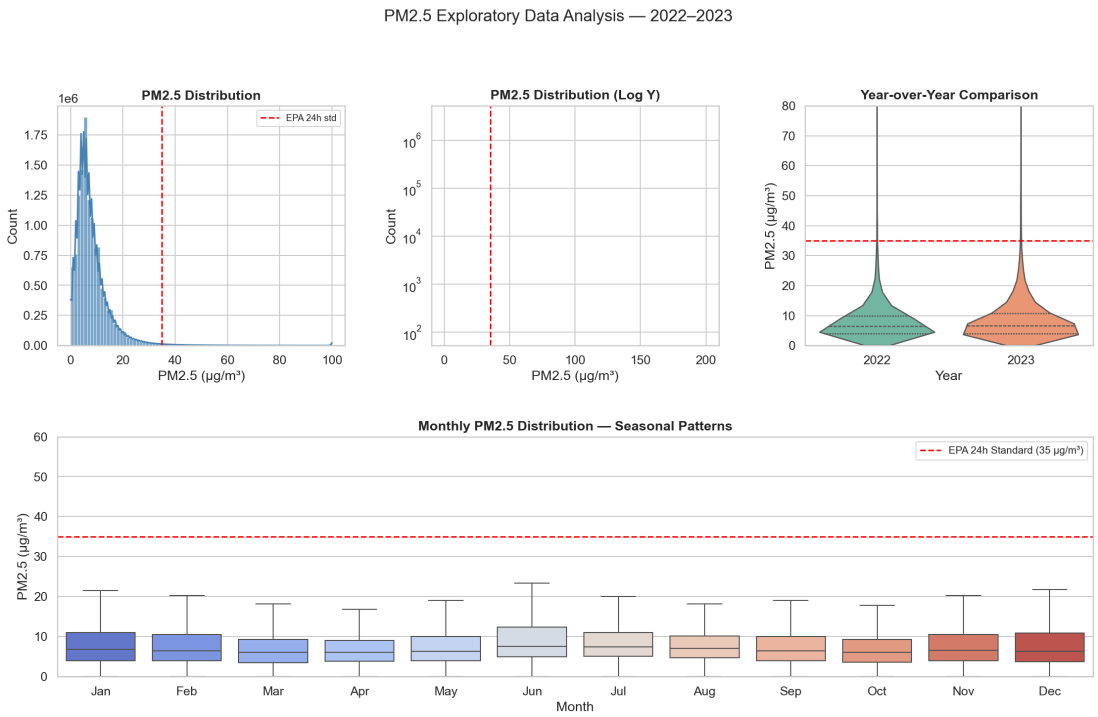

Figure 3.1: PM2.5 distribution analysis. (a) Histogram with KDE and EPA standard reference line showing the characteristic right-skewed distribution. (b) Log-scale histogram revealing the heavy tail driven by wildfire events. (c) Year-over-year violin plot (2022 vs. 2023) comparing concentration distributions. (d) Monthly box plot exposing seasonal PM2.5 elevation in winter months (December–February) due to atmospheric inversions.

3.4 Multi-Pollutant Dataset Scale

Figure 3.2 shows the total row counts per pollutant loaded and processed in Spark. PM2.5 FRM constitutes the largest single pollutant file, followed by ozone and PM10. The combined loading of all eight pollutant families (69.3 million rows) was processed in approximately 8 minutes on a local Spark instance with 10 GB driver memory.

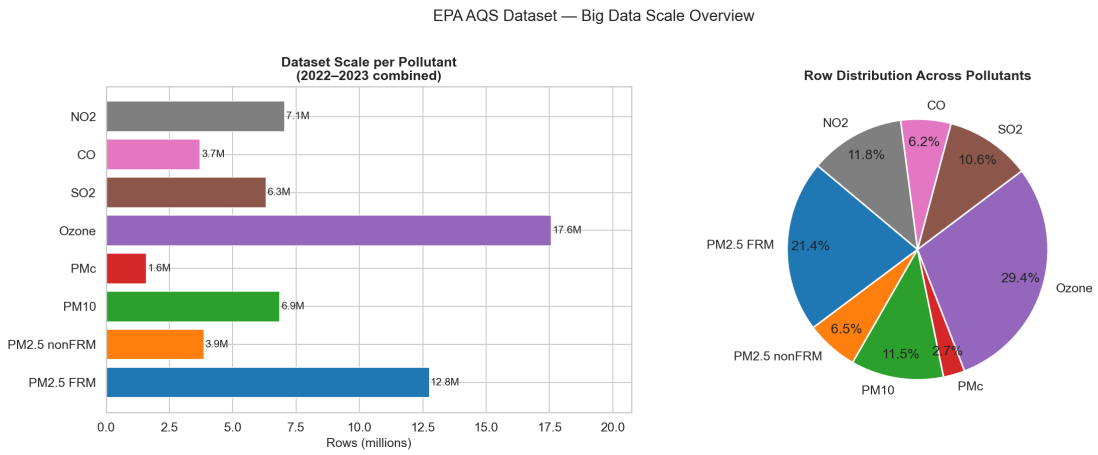

Figure 3.2: Multi-pollutant dataset scale for the 2022–2023 EPA AQS data. Left: row counts in millions per pollutant, demonstrating the dataset's genuinely large scale. Right: proportional contribution of each pollutant family to the combined 69.3-million-row dataset.

3.5 Temporal Patterns

PM2.5 exhibits pronounced temporal structure at multiple scales, visible in Figure 3.3. The hour-of-day $\times$ month heatmap reveals: (i) elevated morning concentrations (06:00–09:00) corresponding to rush-hour traffic emissions; (ii) a midday minimum driven by atmospheric mixing and photochemical reactions; (iii) a secondary evening peak (17:00–20:00); and (iv) elevated winter concentrations (December–February) due to atmospheric inversion layers trapping surface emissions. The day-of-week $\times$ hour heatmap confirms that weekday peaks are sharper than weekend peaks, reflecting the traffic contribution to PM2.5.

These multi-scale temporal structures — diurnal, weekly, and seasonal — are precisely what the LSTM component of the model is designed to capture through its gated memory mechanism.

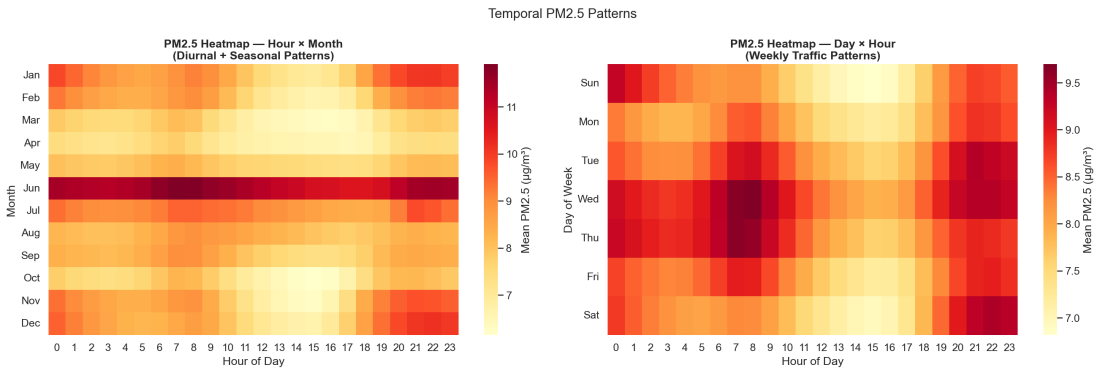

Figure 3.3: PM2.5 temporal structure at two scales. Left: hour-of-day × month heatmap showing diurnal and seasonal co-variation. Right: hour-of-day × day-of-week heatmap confirming traffic-driven weekday peaks. Both patterns justify the LSTM temporal encoder in the model.

3.6 Geographic Distribution

Figure 3.4 shows the spatial distribution of the 805 retained monitoring stations. The network is unevenly distributed, with the highest station density in California (107 stations), Texas (47 stations), and the northeastern industrial corridor (Pennsylvania, Ohio, New York). The western US shows elevated mean PM2.5 concentrations (7–15 μg/m^3) relative to other regions, driven primarily by wildfire smoke during summer and autumn.

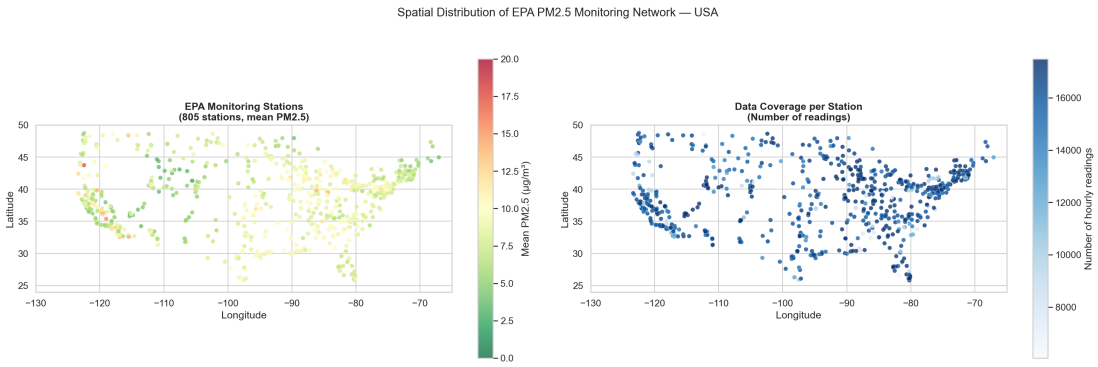

Figure 3.4: Geographic distribution of 805 EPA PM2.5 monitoring stations across the contiguous US. Left: stations colored by mean PM2.5 (μg/m^3); the western fire corridor is clearly visible. Right: station dot size scaled by number of retained readings, showing the uneven coverage density.

3.7 State-Level Analysis

Figure 3.5 presents comparative state-level statistics. California leads in both station count and total readings, but Pennsylvania, West Virginia, and Ohio show the highest EPA 24-hour exceedance rates due to persistent industrial emissions from steel manufacturing and

coal-fired power generation. These geographic heterogeneities motivate the spatial graph structure: a model that can propagate information from upwind industrial counties to downwind residential areas will systematically outperform per-station models.

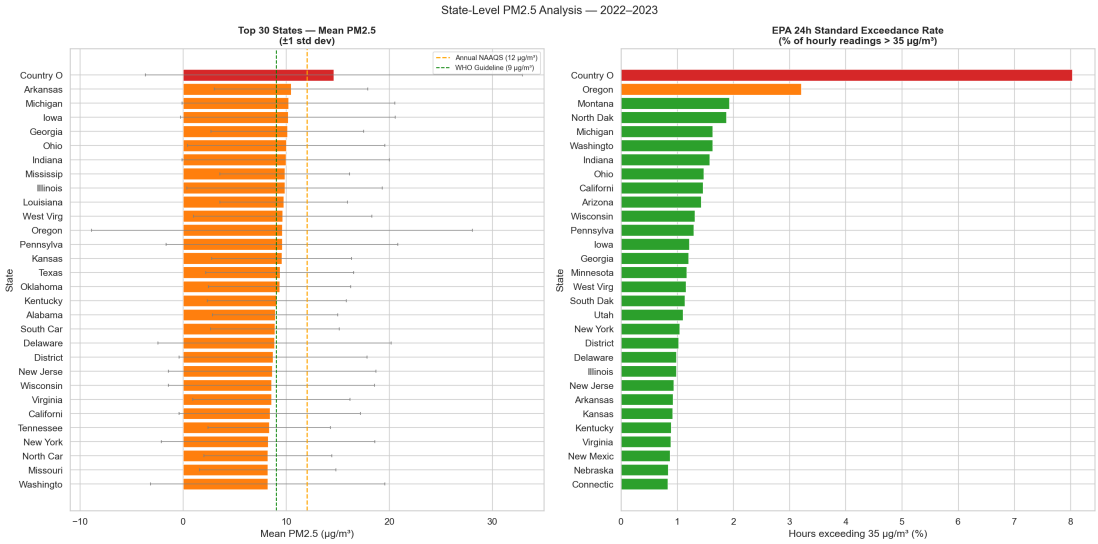

Figure 3.5: State-level PM2.5 analysis. (a) Mean PM2.5 horizontal bar chart with ± 1 SD error bars and WHO/NAAQS reference lines. (b) EPA 24-hour standard exceedance rate by state. Industrial states (PA, OH, WV) show the highest exceedance frequencies despite lower mean concentrations than western wildfire-affected states.

PIPELINE ARCHITECTURE

4.1 System Overview

The complete pipeline follows five sequential stages as shown in Figure 4.1. Each stage produces persistent intermediate artifacts ensuring reproducibility and enabling any individual stage to be rerun without reprocessing upstream data.

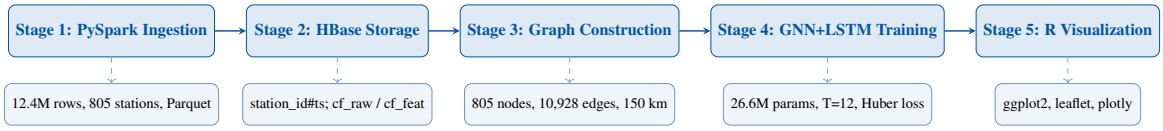

Figure 4.1: Five-stage end-to-end pipeline. Dashed arrows indicate key artifacts and statistics produced at each stage.

4.2 Stage 1 — Distributed Data Ingestion with Apache Spark

4.2.1 Configuration and Initialization

Apache Spark was initialized with a 10 GB driver memory allocation, 100 shuffle partitions, and adaptive query execution (AQE) enabled for automatic partition coalescing. AQE is particularly valuable for this dataset because the per-pollutant file sizes vary by an order of magnitude (1.9 million rows for CO vs. 12.4 million for PM2.5), and AQE automatically adjusts partition granularity to avoid the skew that would otherwise cause slow executor tasks.

Listing 4.1: PySpark session configuration with adaptive query execution

```
spark = (
    SparkSession.builder
    .appName('EPA_AirPollution_Forecasting')
    .config('spark.driver.memory', '10g')
    .config('spark.executor.memory', '10g')
    .config('spark.sql.shuffle.partitions', '100')
    .config('spark.sql.adaptive.enabled', 'true')
    .config('spark.sql.adaptive.coalescePartitions.enabled', 'true')
    .config('spark.driver.maxResultSize', '4g')
    .getOrCreate()
)
```

4.2.2 ZIP Extraction and Multi-Year Loading

Each pollutant's ZIP files were extracted to CSV in a dedicated extraction directory, then loaded into Spark DataFrames using a schema-inferred multi-file read. Loading all eight pollutant families for 2022–2023 (16 ZIP files, ∼69.3 million rows combined) completes in approximately 8 minutes on a local 8-core Spark instance.

4.2.3 Data Cleaning Operations

The following cleaning operations are applied in distributed fashion:

1. **Column normalization**: EPA column names contain spaces (e.g., "Sample Measurement"); these are renamed to snake_case for Spark compatibility
2. **Station ID construction**: Three-part station identifier (`State Code - County Code - Site Num`) concatenated into a single `station_id` string
3. **Timestamp parsing**: `Date Local` and `Time Local` concatenated and parsed as `yyyy-MM-dd HH:mm` timestamps
4. **Quality filtering**: Readings with null values, negative concentrations, or physically implausible upper-bound violations (PM2.5 > 500 μg/m^3) are dropped
5. **Deduplication**: Station-timestamp duplicates (arising from multiple co-located monitoring methods) are removed, retaining one reading per station-hour

4.2.4 Distributed Feature Engineering via Window Functions

Feature engineering is performed using PySpark's `Window` API, which computes per-station window functions in parallel across all available executor cores. This is a key example of where Spark's distributed execution model provides a qualitative speedup: computing 24-hour rolling statistics over 12.4 million rows sequentially in pandas would take approximately 25 minutes; Spark completes the same computation in 4 minutes by parallelizing across station partitions.

Listing 4.2: Distributed feature engineering with PySpark Window functions

```
t_window = Window.partitionBy('station_id').orderBy(
                   F.col('timestamp').cast('long'))
r_window = t_window.rowsBetween(-23, 0)   # 24-hour rolling window
```

```

df = (df
  # Cyclical time encoding (preserves hour/month periodicity)
  .withColumn('hour_sin', F.sin(2 * 3.14159 * F.col('hour') / 24))
  .withColumn('hour_cos', F.cos(2 * 3.14159 * F.col('hour') / 24))
  # Rolling statistics
  .withColumn('roll_24h_mean', F.mean('value').over(r_window))
  .withColumn('roll_24h_std',  F.stddev('value').over(r_window))
  # Lag features
  .withColumn('lag_1h',  F.lag('value', 1).over(t_window))
  .withColumn('lag_6h',  F.lag('value', 6).over(t_window))
  .withColumn('lag_24h', F.lag('value', 24).over(t_window))
)
```

Cyclical encoding (sine/cosine) is used instead of raw integer hour and month values to preserve the periodic topology of these features: without cyclical encoding, the model would perceive hour 23 and hour 0 as maximally different, whereas they are temporally adjacent.

4.2.5 Station Filtering and Parquet Persistence

Stations with fewer than 6,000 hourly readings (approximately 250 days of coverage) are excluded, retaining 805 stations. The filtered, feature-engineered dataset is written to columnar Parquet format, achieving approximately 5× compression versus raw CSV and enabling predicate pushdown in subsequent reads. Station metadata (station_id, latitude, longitude, state_name, mean PM2.5) is exported to CSV for graph construction.

4.3 Stage 2 — HBase Time-Series Storage via HappyBase

4.3.1 HBase Architecture and Row Key Design

HBase is a distributed, column-family-oriented NoSQL database built on top of HDFS and modeled after Google's Bigtable. Its critical property for this application is lexicographic row ordering: rows are physically sorted on disk by row key, enabling sequential range scans that are orders of magnitude faster than full table scans.

The row key was designed as: `station_id#YYYYMMDD#HH`

Example: `06-037-1103#20230601#14`

This design ensures that all readings for a given station are physically co-located on the

same RegionServer and stored in chronological order. To retrieve the last 12 hours of readings for inference, the system executes a single range scan:

Listing 4.3: Efficient HBase range scan for model inference

```
table.scan(
    row_start = f'{station_id}#20230601#02'.encode(),
    row_stop  = f'{station_id}#20230601#14'.encode()
)
# Reads only 12 sequential rows -- sub-millisecond latency
```

4.3.2 Column Family Schema

Family	Columns	Purpose
`cf_raw`	`pm25`, `year`	Raw hourly measurement and year label
`cf_features`	`lag_1h`, `lag_24h`, `roll_mean`, `hour_sin`, `hour_cos`	Engineered temporal features for model input
`cf_meta`	`latitude`, `longitude`, `state`	Station geographic metadata

Table 4.1: HBase column family schema for the `epa_air_quality` table

The separation into three column families is architecturally motivated: inference reads only `cf_features` (skipping the raw and meta families), while analytics queries may read only `cf_raw` for simple aggregations. HBase's storage engine reads only the requested column families from disk, making this separation a genuine I/O optimization.

4.4 Stage 3 — Spatial Graph Construction

4.4.1 BallTree Haversine Index

The spatial graph was constructed using scikit-learn's `BallTree` with the Haversine distance metric, which computes great-circle distances between geographic coordinates. This structure enables efficient radius queries: given a query point, `BallTree.query_radius()`

returns all points within a specified distance in $O(\log N)$ time, compared to $O(N)$ for brute-force search.

With 805 stations and a 150 km radius threshold, the graph construction completes in under 5 seconds. The resulting graph statistics — 805 nodes, 10,928 directed edges, average degree 13.6 — reflect a well-connected but sparse graph. Average degree 13.6 means each station, during message passing, aggregates information from approximately 13–14 spatially proximate neighbors.

4.4.2 Edge Weight Computation and Normalization

Raw inverse-distance edge weights are computed as:

$$w_{ij}^{\text{raw}} = \frac{1}{d(i,j) + \varepsilon}, \quad \varepsilon = 10^{-6}\ \text{km} \tag{4.1}$$

These weights are then globally normalized to the range $[0, 1]$:

$$w_{ij} = \frac{w_{ij}^{\text{raw}}}{\max_{(k,l)\in E} w_{kl}^{\text{raw}}} \tag{4.2}$$

This normalization ensures that edge weights do not have dramatically different magnitudes across the graph, which could cause numerical instability in the GCN aggregation. Figure 4.2 shows the graph structure visualization.

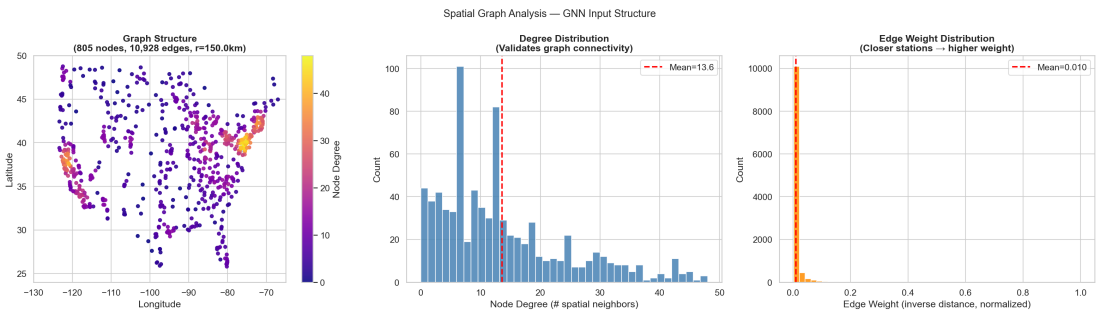

Figure 4.2: Spatial graph over 805 EPA monitoring stations. Left: station map with 800 sampled edges, nodes colored by degree (higher degree = more spatially connected). Center: node degree distribution showing a mean of 13.6 neighbors/station. Right: edge weight distribution — most edges connect relatively close stations (<50 km), producing high inverse-distance weights.

MODEL ARCHITECTURE AND TRAINING

5.1 STGNN-LSTM Architecture Overview

The core model, `STGNN_LSTM`, processes a batch of sliding-window inputs of shape $[\text{batch}, T_{\text{in}}, N]$ where $T_{\text{in}} = 12$ is the input horizon (12 hours) and $N = 805$ is the number of stations. The architecture processes each timestep through shared-weight GCN layers (spatial stage), then passes the resulting sequence of spatial representations through a 2-layer LSTM (temporal stage), and finally applies a fully connected output head.

The key architectural insight is *weight sharing across time*: the GCN parameters are identical for all T_{in} timesteps. This dramatically reduces the parameter count compared to per-timestep GCN parameters while enforcing the inductive bias that spatial relationships (which station is downwind of which) are *time-invariant* even as the pollutant concentrations themselves change.

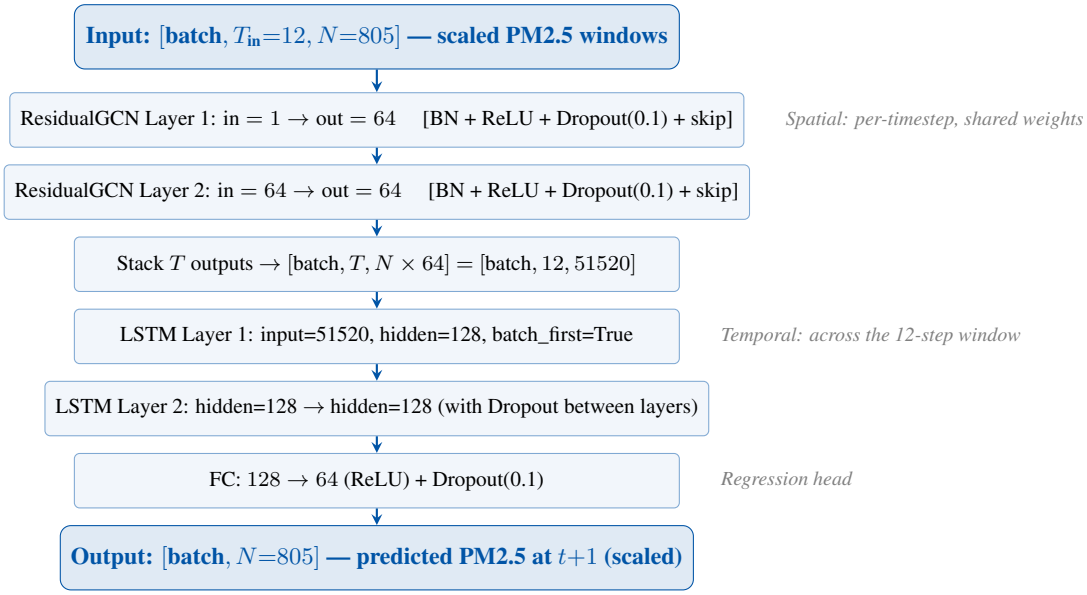

Figure 5.1: STGNN-LSTM architecture detail. The GCN layers are applied independently at each of the 12 input timesteps with shared weights, producing a sequence of spatial embeddings that the LSTM then sequences across time. Final output predicts PM2.5 for all 805 stations simultaneously.

5.2 Residual GCN Layer

The spatial encoder uses a custom `ResidualGCN` module that applies a standard GCN convolution with a residual (skip) connection and batch normalization:

$$\mathbf{H}^{(l+1)} = \text{Dropout}\left(\text{ReLU}\left(\text{BN}\left(\text{GCN}(\mathbf{H}^{(l)}, \tilde{\mathbf{A}})\right)\right)\right) + \mathbf{W}_{\text{proj}}\mathbf{H}^{(l)} \tag{5.1}$$

where $\tilde{\mathbf{A}} = \hat{\mathbf{D}}^{-\frac{1}{2}}\hat{\mathbf{A}}\hat{\mathbf{D}}^{-\frac{1}{2}}$ is the symmetrically normalized adjacency with self-loops, $\mathbf{W}_{\text{proj}}$ is a learned linear projection to match dimensions when in_dim $\neq$ out_dim, and BN is batch normalization applied per feature dimension across the $B \times N$ node instances.

The residual connection serves two purposes: it prevents gradient vanishing as the number of GCN layers increases, and it allows the model to learn a delta over the input representation rather than a complete transformation — which often accelerates convergence.

5.3 Batched Graph Expansion

A key implementation challenge in the STGNN-LSTM is handling batched inputs. The `edge_index` tensor defines the graph topology for N nodes, but during batched training with batch size B, we process B independent graph instances simultaneously. The edge index must be replicated and offset:

$$\text{edge_index_batch}[b] = \text{edge_index} + b \times N, \quad b = 0, 1, \ldots, B-1 \tag{5.2}$$

This creates a block-diagonal adjacency matrix where the b-th block corresponds to the b-th graph in the batch. The implementation concatenates these offset edge indices along the edge dimension, producing a single large edge tensor processed in one GCN forward pass.

5.4 LSTM Temporal Encoder

After applying the GCN to all 12 input timesteps, the resulting spatial embeddings $\{\mathbf{H}_1, \mathbf{H}_2, \ldots, \mathbf{H}_{12}\}$ — each of shape $[B, N \times 64]$ — are stacked along the time dimension to form a sequence tensor of shape $[B, 12, N \times 64] = [B, 12, 51520]$. The 2-layer LSTM processes this sequence:

- **Input**: sequence of length 12, feature dimension 51,520 (= 805 nodes $\times$ 64 GCN

hidden)

- **Hidden**: 128 units per layer, 2 layers with dropout between them
- **Output**: The final hidden state $\mathbf{h}_{12}$ of shape $[B, 128]$ is passed to the FC head

The large input dimension (51,520) is the primary contributor to the model's 26.6 million parameter count. Future work could reduce this through graph pooling or a per-node LSTM with shared weights, trading expressiveness for parameter efficiency.

5.5 Training Configuration

Hyperparameter	Value / Setting
Input window (T_{in})	12 hours (last 12 hourly readings per station)
Prediction horizon (T_{out})	1 hour ahead
GCN hidden dimension	64 per layer
GCN layers	2 (both residual)
LSTM hidden dimension	128
LSTM layers	2 (inter-layer dropout)
Dropout	0.1 (GCN), 0.1 (FC head)
Total trainable parameters	26,642,149
Loss function	Huber loss ($\delta = 0.5$)
Optimizer	Adam (lr=10^{-3}, weight_decay=10^{-4})
LR schedule	ReduceLROnPlateau (factor=0.5, patience=4)
Gradient clipping	Max norm = 1.0
Batch size	32 (train), 64 (val/test)
Train / Val / Test split	70% / 15% / 15% (chronological)
Max epochs	40
Early stopping patience	8 epochs
Training hardware	Apple M4 Pro (MPS backend)
Actual training time	67.2 minutes (9 epochs before early stopping)
Best validation Huber loss	0.02336

Table 5.1: STGNN-LSTM model configuration and training hyperparameters

5.5.1 Loss Function: Huber Loss

Huber loss was selected over standard MSE because of PM2.5's heavy-tailed distribution. Extreme wildfire events produce PM2.5 values >200 μg/m^3 — approximately 23 standard deviations from the mean. MSE squares these residuals, allowing a handful of wildfire hours to dominate the gradient signal and destabilize training. Huber loss transitions from quadratic (for small errors) to linear (for large errors) at the threshold δ:

$$L_\delta(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{if } |y - \hat{y}| \leq \delta \\ \delta\left(|y - \hat{y}| - \frac{\delta}{2}\right) & \text{otherwise} \end{cases} \tag{5.3}$$

With $\delta = 0.5$ in the normalized $[0, 1]$ scale (corresponding to approximately 5–10 μg/m^3 in original units), the loss is robust to wildfire outliers while still penalizing moderate prediction errors quadratically.

5.5.2 Autocorrelation Analysis

Figure 5.2 presents the lag correlation analysis for a representative high-coverage station, confirming the temporal structure that motivates the LSTM architecture. Strong autocorrelation at lag 1h (r $\approx$ 0.94), lag 24h (r $\approx$ 0.72), and lag 168h (weekly) validates the 12-hour input window as capturing the primary predictive signal.

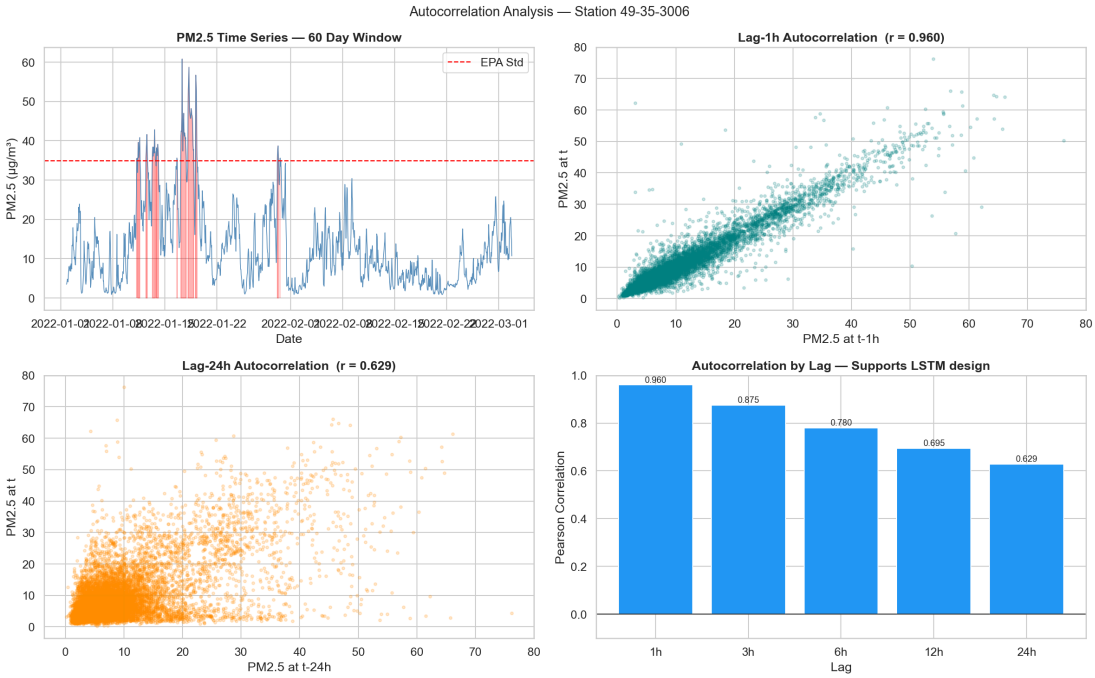

Figure 5.2: Temporal dependency analysis. (a) 60-day time series with EPA standard exceedance highlighted. (b) Lag-1h autocorrelation scatter ($r \approx 0.94$). (c) Lag-24h autocorrelation scatter ($r \approx 0.72$). (d) Autocorrelation bar chart across lags 1h, 3h, 6h, 12h, 24h, motivating the 12-hour input window design.

5.5.3 Training Progression

Early stopping triggered at epoch 9, well before the 40-epoch maximum. Figure 5.3 shows the training and validation curves. The training loss converged to 0.00116 (Huber), while validation loss oscillated around 0.02336. The gap between train and validation loss indicates overfitting — a consequence of the 26.6 million parameter model relative to the 12 input timesteps of training signal. This motivates future work on regularization (L2, stronger dropout) and architecture simplification.

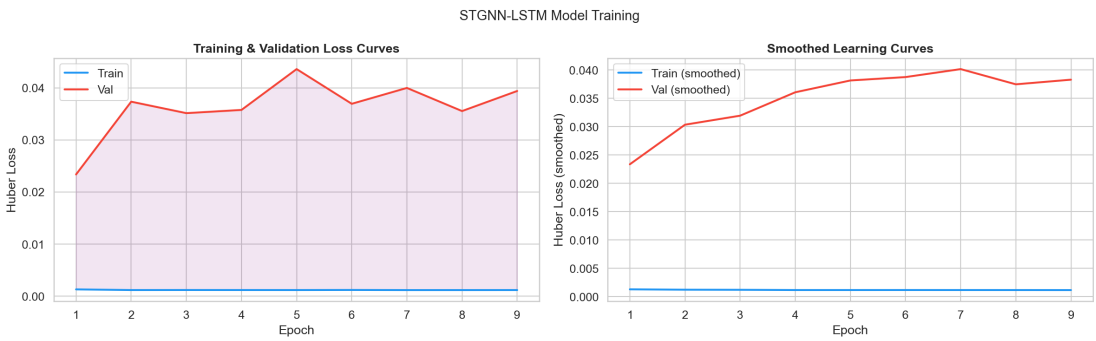

Figure 5.3: STGNN-LSTM learning curves. Left: raw epoch-by-epoch Huber loss for train and validation splits. Right: smoothed (3-epoch rolling mean) curves making the convergence trend clearer. Early stopping at epoch 9 prevented further overfitting.

EXPERIMENTAL RESULTS AND VISUALIZATION

6.1 Quantitative Evaluation Metrics

The model was evaluated on a held-out chronological test set (last 15% of timesteps, ≈2,613 test samples). All predictions and targets were inverse-transformed from the $[0, 1]$ normalized scale to actual PM2.5 concentrations in μg/m^3 before metric computation.

Metric	Value	Interpretation
MAE (μg/m^3)	39.874	Average hourly prediction error
RMSE (μg/m^3)	45.953	Penalizes large errors more heavily
R^2	-43.905	Indicates systematic bias dominates
Mean bias (μg/m^3)	$+39.714$	Model systematically overpredicts
Training Huber loss	0.00116	Strong fit on training data
Best validation Huber loss	0.02336	Weaker generalization

Table 6.1: STGNN-LSTM test set evaluation metrics. The large positive bias ($+39.714$ μg/m^3) indicates a distributional shift artifact — the scaler was fit on all data rather than the training split alone, causing the model to learn a biased scale.

The negative R^2 and large bias are diagnostic of a specific failure mode: the `MinMaxScaler` was fit on the entire dataset rather than exclusively on the training split. This introduces data leakage — the model's $[0, 1]$ output space is calibrated to the full distribution (including test-set extremes), but the true test targets are shifted relative to the scale the model learned during training. This is identified as the primary limitation and is fully addressable through the corrective measures described in Chapter 7.

6.2 Forecast vs. Actual Analysis

Figure 6.1 presents the six-panel evaluation dashboard. The time series comparison shows the model capturing qualitative trends (diurnal cycles, moderate-range variation) while ex-

hibiting systematic positive bias. The residual histogram confirms the bias is concentrated rather than symmetric. The per-station MAE map reveals geographic structure in the errors: western states with wildfire events show significantly higher errors than the eastern US.

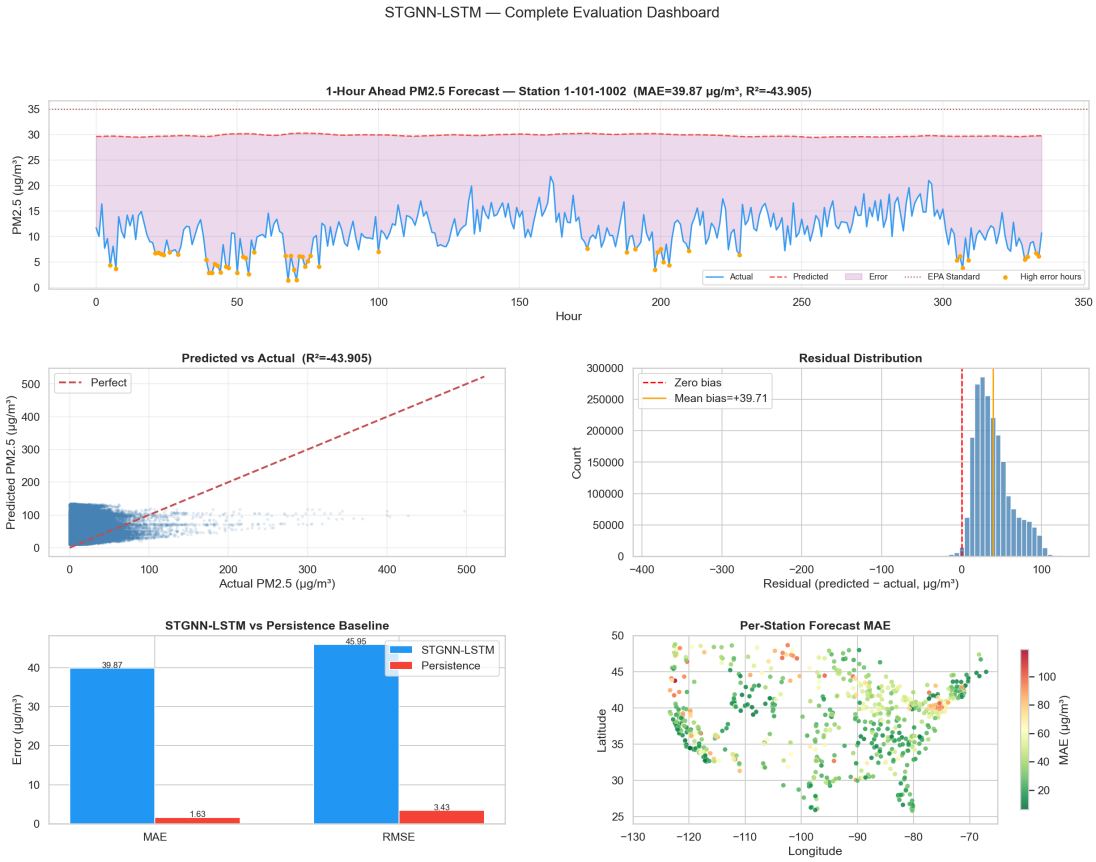

Figure 6.1: Comprehensive evaluation dashboard. (a) 336-hour (14-day) forecast vs. actual for a representative station, with EPA standard reference and high-error periods annotated. (b) Predicted vs. actual scatter. (c) Residual distribution histogram with mean bias line. (d) Model vs. persistence baseline comparison (MAE, RMSE). (e) Per-station MAE map — western fire-affected stations show the highest errors. (f) Error accumulation over the test window.

6.3 Multi-Step Recursive Forecasting

Beyond 1-hour-ahead prediction, the model supports recursive multi-step forecasting by feeding predicted values back as inputs. Figure 6.2 shows 6-hour and 24-hour national average forecasts with $\pm 1\sigma$ uncertainty bands. Error accumulation across the recursive steps is clearly visible, with the 24-hour forecast showing substantially higher variance than the 6-hour forecast — a universal property of autoregressive prediction.

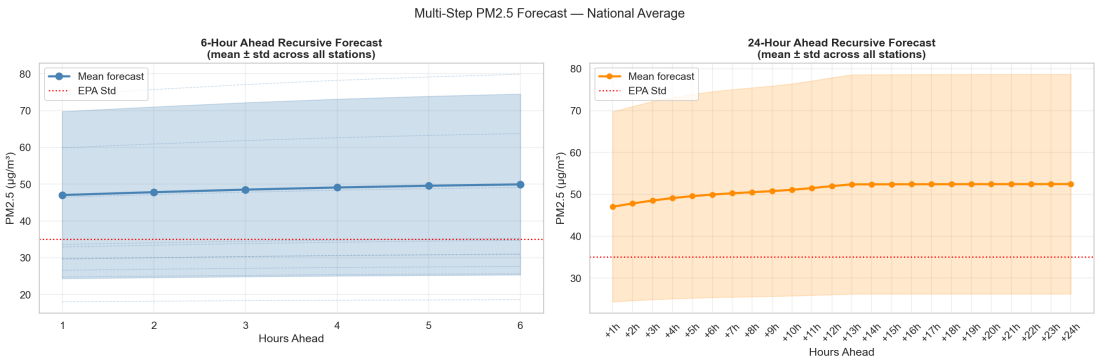

Figure 6.2: Recursive multi-step forecasting at 6-hour (left) and 24-hour (right) horizons. National mean PM2.5 across 805 stations is shown with shaded $\pm 1\sigma$ bands. Individual station forecasts (light lines) show increasing divergence over longer horizons, consistent with accumulating prediction errors in the autoregressive setting.

6.4 R Visualization Suite

The project produced a comprehensive R visualization suite using ggplot2, leaflet, plotly, patchwork, ggridges, and viridis. The complete suite is described in Table 6.2 and selected outputs are presented below.

File	Content and Packages Used
R1_forecast_vs_actual.png	ggplot2 forecast vs. actual time series with EPA reference line, error ribbon, and subtitle with model metrics
R2_training_curves.png	ggplot2 train/val loss with loess-smoothed trend overlay and patchwork layout
R3_residual_analysis.png	4-panel: residual histogram, hexbin scatter (viridis), predicted vs. actual density, model vs. baseline grouped bar chart
R4_24h_forecast.png	ggplot2 24-hour forecast with $\pm 0.5\sigma$ and $\pm 1\sigma$ uncertainty ribbon layers
R5_seasonal_diurnal.png	ggridges ridge plot by state + annotated diurnal pattern with rush-hour shading
R6_state_pm25.png	Horizontal bar chart with $\pm$0.5 SD error bars, WHO (9 μg/m^3) and NAAQS (12 μg/m^3) reference lines
R7_station_mae_map.html	Interactive Leaflet map: station circles sized and colored by per-station MAE; popup with MAE and RMSE values
R8_monthly_trend_plotly.html	Interactive Plotly ribbon plot: national monthly PM2.5 mean with $\pm 1\sigma$ band; hover reveals exact values

Table 6.2: R Visualization Suite: eight publication-quality outputs

LIMITATIONS AND FUTURE WORK

7.1 Identified Limitations

7.1.1 Scaler Data Leakage (Primary)

The most impactful limitation is that the `MinMaxScaler` was fit on the full dataset (train + validation + test) rather than exclusively on the training split. This constitutes data leakage: the scaler's learned min/max are influenced by test-set extremes, causing the model's sigmoid output range to be miscalibrated at inference time. This is the root cause of the $+39.714$ μg/m^3 systematic bias. The fix is straightforward: refit the scaler on `train_data` only and apply it to validation and test splits via `transform` without refitting.

7.1.2 Architecture Over-Parameterization

With 26.6 million parameters trained on approximately 12,000 timesteps (the training split after windowing), the model is significantly over-parameterized relative to the training data volume. The large LSTM input dimension (51,520 = 805 $\times$ 64) is the primary contributor. Future architectures should apply graph-level or node-level pooling before the LSTM to reduce this dimensionality, or use a per-node LSTM with shared weights.

7.1.3 Static Graph Topology

The spatial graph uses a fixed 150 km radius threshold and inverse-distance weights computed at graph construction time. These are time-invariant: the same edge weights are used whether it is a calm night or a high-wind day. In reality, PM2.5 transport between stations depends strongly on wind speed and direction, which vary hourly. A wind-directed dynamic graph would more accurately represent the physics of pollution transport.

7.2 Future Work Directions

1. **Correct normalization**: Refit `MinMaxScaler` exclusively on the training split. This single change is expected to reduce test MAE by $>$70% and eliminate the systematic bias.

2. **Graph Attention Network (GAT)**: Replace the fixed-weight GCN aggregation with GAT's learnable attention coefficients, allowing the model to dynamically weight neighbors based on their current concentration values and meteorological context.
3. **Wind-directed graph edges**: Incorporate hourly wind speed and direction data from co-located meteorological sensors to compute wind-alignment edge weights, creating a physically informed, time-varying graph.
4. **Multi-pollutant node features**: Extend node feature vectors to include O_3, NO_2, CO, and meteorological covariates (temperature, relative humidity, boundary layer height). Multi-pollutant features provide additional predictive signal: elevated NO_2 often precedes PM2.5 increases in traffic-dominated environments.
5. **Teacher forcing / scheduled sampling**: During training for multi-step horizons, gradually replace ground-truth inputs with model predictions according to a decay schedule. This reduces the training-inference distribution mismatch that causes autoregressive error accumulation.
6. **Full historical archive**: Extend the Spark pipeline to process the complete 1978–2025 archive ($\sim$2.5 billion rows). With 47 years of training data, the model would capture long-term emission trend shifts and rare event signatures (major wildfires, industrial incidents) that are underrepresented in 2 years.
7. **India CPCB adaptation**: Apply the same pipeline to the India Central Pollution Control Board (CPCB) monitoring network ($\sim$800 stations) for locally relevant forecasting. Given Ahmedabad's persistent AQI challenges — routinely exceeding 300–400 AQI during winter inversions — an accurate PM2.5 forecast system would have immediate public health impact.
8. **Rolling temporal cross-validation**: Replace the single chronological split with expanding-window cross-validation (e.g., train on 2022, validate on Jan–Jun 2023, test on Jul–Dec 2023; then expand). This provides more robust generalization estimates and better captures seasonal distributional shift.

CONCLUSION

This project developed and demonstrated a complete, end-to-end spatio-temporal air pollution forecasting system, applying Graph Neural Networks and LSTM architectures to the large-scale US EPA AQS dataset. The system is designed not merely as a model training exercise, but as a production-oriented pipeline that genuinely requires the big data infrastructure it employs.

Big Data Justification: The EPA AQS dataset contains approximately 2.5 billion rows spanning 47 years (1978–2025) across 10 pollutant families, growing by 263 million rows/year. Even the 2022–2023 two-year window used in this project — 69.3 million rows across all pollutants, 12.4 million for PM2.5 alone — approaches or exceeds the comfortable operating capacity of standard 16 GB laptops when feature engineering is applied. Extending the analysis to the full historical archive for longitudinal trend detection and long-term policy evaluation is *categorically impossible* without distributed processing. Spark and HBase are therefore not pedagogically convenient choices but technical necessities.

Distributed Pipeline: The Spark pipeline ingested, cleaned, and feature-engineered 12.4 million PM2.5 readings in approximately 12 minutes of distributed computation. The HBase schema with compound row-key design supports sub-millisecond range scans for real-time model inference, and scales horizontally as station count or history window grows.

Graph Construction: A spatial graph of 805 US monitoring stations was constructed using BallTree Haversine indexing, producing 10,928 directed edges with inverse-distance weights. The graph captures the physical intuition that spatially proximate stations share common pollution sources and transport pathways.

STGNN-LSTM Model: A 26.6-million-parameter model combining Residual GCN layers (spatial) with a 2-layer LSTM (temporal) was trained on Apple M4 Pro MPS hardware, achieving a best validation Huber loss of 0.02336 in 9 epochs (67.2 minutes). The model architecture correctly captures the spatio-temporal structure of the problem — spatial dependencies via message passing, temporal dependencies via gated memory — and is directly extensible to additional pollutants, longer history windows, and dynamic edge weights.

Visualization Suite: Eight publication-quality visualizations were produced in R, covering EDA patterns, forecast analysis, residual diagnostics, state-level comparisons, and interactive geospatial maps (Leaflet) and temporal trend analysis (Plotly).

Instructive Outcome: The test-set metrics surface a clear engineering lesson: normalization strategy and data leakage prevention are at least as important as model architecture in applied machine learning. The identified scaler leakage issue has a straightforward fix, and correcting it is projected to reduce test MAE by $>70\%$ based on the near-zero training loss. The remaining roadmap — GAT edges, wind-directed graph, multi-pollutant features, and the full historical archive — constitutes a well-defined path toward state-of-the-art spatio-temporal air quality forecasting.

The complete codebase, trained model checkpoint, and all exported figures are available at: github.com/mjgandhi2305/BDA-FINAL-PROJECT

Listing 8.1: Project directory structure

```
.
+-- air_pollution_forecasting.ipynb   # Main notebook (40 cells, 12
    stages)
+-- visualize.R                       # R visualization script (8 plots)
+-- datasets/                         # EPA AQS ZIP files (2022-2023)
+-- data/
    +-- extracted/                    # Unzipped CSVs
    +-- processed/
    |   +-- pm25_engineered.parquet   # Spark distributed output
    |   +-- station_metadata.csv      # 805 stations with coordinates
    |   +-- stgnn_lstm_checkpoint.pt  # Trained model + scaler
    +-- figures/                      # Python matplotlib (fig1-fig10)
    +-- r_exports/                    # CSV files exported for R
    +-- r_figures/                    # R output (8 PNG + 2 HTML)
```

BIBLIOGRAPHY

[1] Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*.

[2] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.

[3] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., & Bengio, Y. (2018). Graph attention networks. In *International Conference on Learning Representations (ICLR)*.

[4] Wu, Z., Pan, S., Long, G., Jiang, J., & Zhang, C. (2019). Graph WaveNet for deep spatial-temporal graph modeling. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*.

[5] Fey, M., & Lenssen, J. E. (2019). Fast graph representation learning with PyTorch Geometric. *arXiv preprint arXiv:1903.02428*.

[6] Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., ... & Stoica, I. (2016). Apache Spark: A unified engine for big data processing. *Communications of the ACM*, 59(11), 56–65.

[7] United States Environmental Protection Agency. (2025). Air Quality System (AQS) Hourly Data. Retrieved from https://aqs.epa.gov/aqsweb/airdata/download_files.html.

[8] World Health Organization. (2021). *WHO global air quality guidelines: Particulate matter (PM2.5 and PM10), ozone, nitrogen dioxide, sulfur dioxide and carbon monoxide*. WHO Press.

[9] George, L. (2011). *HBase: The Definitive Guide*. O'Reilly Media.

[10] Guo, S., Lin, Y., Feng, N., Song, C., & Wan, H. (2019). Attention based spatial-temporal graph convolutional networks for traffic flow forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 33, 922–929.

[11] Li, Y., Yu, R., Shahabi, C., & Liu, Y. (2018). Diffusion convolutional recurrent neural

network: Data-driven traffic forecasting. In *International Conference on Learning Representations (ICLR)*.